On the Limits of Consensus under Dynamic Availability and Reconfiguration

Joachim Neu[1], Javier Nieto[2], and Ling Ren[2]

[1]a16z Crypto Research, jneu@a16z.com
[2]University of Illinois at Urbana-Champaign, {jmnieto2,renling}@illinois.edu

Abstract

Proof-of-stake blockchains require consensus protocols that support Dynamic Availability and Reconfiguration (so-called DAR setting), where the former means that the consensus protocol should remain live even if a large number of nodes temporarily crash, and the latter means it should be possible to change the set of operating nodes over time. State-of-the-art protocols for the DAR setting, such as Ethereum, Cardano's Ouroboros, or Snow White, require unrealistic additional assumptions, such as social consensus, or that key evolution is performed even while nodes are not participating. In this paper, we identify the necessary and sufficient adversarial condition under which consensus can be achieved in the DAR setting without additional assumptions. We then introduce a new and realistic additional assumption: honest nodes dispose of their cryptographic keys the moment they express intent to exit from the set of operating nodes. To add reconfiguration to any dynamically available consensus protocol, we provide a bootstrapping gadget that is particularly simple and efficient in the common optimistic case of few reconfigurations and no double-spending attempts.

1 Introduction

Today's distributed systems are evolving from static, tightly coordinated deployments into open, decentralized environments with dynamic availability and reconfigurable membership. For instance, classical consensus protocols such as PBFT [CL99] assume a fixed set of always-online servers; which is in stark contrast to modern proof-of-stake (PoS) blockchains, where operators may frequently go offline, join, or leave the network. To this end, current literature has studied two orthogonal dimensions of changing participation.

First, the idea of *reconfigurable* membership in distributed systems came early on as an additional constraint in adding, removing, or replacing nodes after initialization of the protocol. Whereas classical consensus is defined on a single fixed *membership set*, reconfigurable systems are comprised of a sequence of membership sets where a set of nodes make consensus decisions for some time before the next set of nodes take over [LMZ09, LMZ10, SRMJ12, OO14, BSA14, JM14].

Second, and orthogonal to reconfiguration, is the idea of *dynamic availability*, which was pioneered by the Bitcoin protocol [Nak09] to allow unknown levels of participation in the form of computational hash power. Bitcoin does not require a fixed quorum of nodes to make decisions. Instead, the protocol relies on the assumption that honest nodes control the majority of the computational power. Therefore, consensus decisions are inherently tied to the relative levels of participation (i.e., the proportion of honest versus adversarial computational power) at any given time. Pass and Shi [PS17] first formalized this notion as the sleepy model, which has also been referred to as the dynamic availability model in the literature [NTT21, DNTT24]. The model considers a *fixed* set of nodes, where, throughout execution, some honest nodes may be *sleepy*, or unavailable, and do not participate in the protocol. Sleepy nodes can be viewed as crashed nodes that may recover later. However, the sleepy model is not to be confused with mixed fault models [GP92, CMSK07, ADKS22, GDCL22], because the sleepy model allows for an arbitrary amount of sleepy nodes, whereas mixed fault models put a bound on the total number of Byzantine and crash-recovery faults.

It is then natural to combine the two settings into the *Dynamic Availability and Reconfiguration* (DAR) model. While prior works have explored models that appear similar to DAR [DPS19, BGK$^+$18, BHK$^+$20], they rely on assumptions and modeling choices that are hard to justify, as we elaborate below.

Snow White [DPS19] and PoS Ethereum [BHK$^+$20] rely on the assumption of *social consensus*, also known as *weak subjectivity*, which states that newly joining nodes obtain a recent checkpoint of the system from some trusted source. In practice, this mechanism must continuously achieve consensus on an up-to-date sequence of checkpoints so that outdated nodes can determine the latest state to correctly participate in the consensus protocol. The common rationale for social consensus is that newly joining nodes must anyway obtain a correct protocol implementation from some trusted source; thus, obtaining a recent checkpoint of the blockchain state appears to impose little additional requirement. However, this seemingly minor requirement is at odds with the core principles and established motivation of distributed consensus. While all consensus protocols require some initial common knowledge [HM84] (i.e., agreed-upon states), such as the protocol description itself and few public parameters (like the genesis block in Bitcoin), the role of the consensus protocol is to process impromptu inputs subsequently and continuously to expand the agreed-upon state. Social consensus is an external mechanism that provides consensus on a growing sequence of checkpoints to external observers. It therefore seems unsatisfactory, or even circular, for a consensus protocol to assume the existence of an external consensus mechanism in order to solve consensus.

Ouroboros Genesis [BGK$^+$18] is a longest-chain consensus protocol designed for a model that appears similar to DAR[1] and does not rely on social consensus. However, upon

[1]Ouroboros Genesis [BGK$^+$18] uses the term dynamic availability to refer to what we define as the DAR model. We use the term dynamic availability to only refer to the availability/sleepy status of nodes, and we treat reconfiguration (membership changes) as a separate aspect of the model.

Table 1: A comparison of protocols in the DAR model

Protocol	**Type**	**Corruption Threshold**	**No Social Consensus**	**Tolerates Majority Asleep**
Bitcoin [Nak09]	PoW	Honest Majority	✔	✔
Snow White [DPS19]	PoS	Honest Majority	✘	✔
Ethereum [BHK+20]	PoS	Honest Majority	✘	✔
Ouroboros [BGK+18]	PoS	Honest Majority	✔	✘
Algs. 1 and 2	PoS	SR-HM (Def. 4)	✔	✔

closer inspection, their model deviates from DAR by requiring sleepy nodes to perform *key evolution* to maintain forward security. Nodes that fail to do so are counted as adversarial. However, this requirement seems to undermine the very notion of dynamic availability: in the dynamic availability model, a majority of nodes (stakeholders) may not be interested in taking part in the consensus protocol at all. These nodes would not perform any protocol actions. Counting these nodes as adversarial forces the assumption that the majority of nodes are awake and honest, effectively reducing the model to static availability.

In contrast, in this work, we make no additional assumptions on what is common knowledge beyond the protocol description and the initial membership set. We also do not assume nodes perform any actions during their absence. We consider two models for the behavior of exiting nodes during reconfiguration. The first considers reconfigurations where nodes are removed from the membership set without any required participation or authorization on their part. We refer to this minimal model simply as the *DAR model*, as it captures the most general reconfiguration setting. We compare our protocol in this setting with the aforementioned protocols in Tab. 1. The second considers reconfigurations where nodes are required to execute a *sign-off* procedure before exiting. This is typical of PoS systems in which nodes must sign a transaction to transfer stake. We refer to this stronger variant as the *DAR with sign-off* model. Under this model, we give a protocol with much better efficiency over the one in the DAR model when only a small fraction of members are reconfigured per epoch and no double-spending transactions are observed, which is often the case in real-world PoS systems.

Main Results.

- We formalize the necessary condition for solving consensus in the DAR model as *simulation-resistant honest majority* (SR-HM, Def. 4, see Thm. 3.2).
- We propose a generic bootstrapping gadget that transforms any consensus protocol secure in the dynamically available and non-reconfigurable setting into a consensus protocol

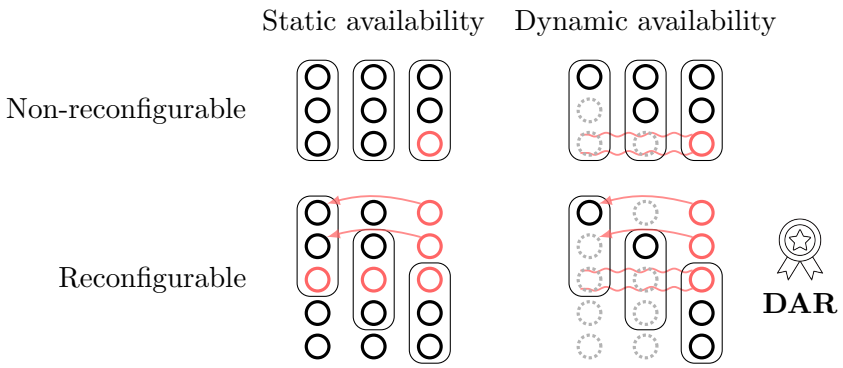

Figure 1: Models of reconfiguration and availability across three rounds. Here, "○" depicts awake and honest nodes, "○" are Byzantine nodes, and "○" are sleepy nodes. "←" depicts long-range attacks, and "≈" depicts backward simulation.

secure in the baseline DAR setting under the SR-HM assumption (see Thm. 3.1), thereby establishing it as the sufficient condition for solving consensus in the DAR model.

- We propose an additional generic bootstrapping gadget with a novel and simple good-case path that leverages signed membership changes to achieve consensus in the *DAR with sign-off* model (see Thm. 4.1).

1.1 Overview

We briefly describe the difficulties in solving consensus under the different combinations of availability and reconfiguration. This will aid in our reasoning in the DAR model. We categorize models along two orthogonal dimensions: (1) availability—static vs. dynamic, and (2) membership—non-reconfigurable vs. reconfigurable. We label each model accordingly, such as "Dynamic Availability and Reconfiguration (DAR)" or "Dynamic Availability and Non-Reconfiguration (Sleepy Model)." We summarize the scenarios in Fig. 1.

Static Availability and Non-Reconfiguration. These systems consider a fixed set of nodes where honest nodes are always available. Classic protocols such as PBFT [CL99] operate in this setting. Many protocols in this setting utilize digital signatures to bind nodes to the messages they sent. Once a node receives sufficiently many signed messages, the node can construct a quorum *certificate*, a transferable proof that a particular event, or decision, has occurred with sufficient backing.

Static Availability and Reconfiguration. These systems largely operate in the same manner as the classic protocols above. There is only a slight tweak in that the certificates

produced throughout the protocol must be verified with respect to the corresponding membership set. Therefore, verification usually starts from the last known membership set and *iteratively* verifies every consensus decision and reconfiguration up to the most current output of the protocol.

Reconfiguration brings a subtlety known as the *long-range attack* [DPP19]. It may be unrealistic to expect an honest node that has since exited the system to maintain its honesty indefinitely. Thus, it is most desirable to place fault bounds only on the latest set of members. However, nodes that have exited the system still possess their cryptographic signing keys. If enough of them are corrupted by the adversary, the adversary can generate certificates for an alternative sequence of past decisions, essentially *rewriting history* in a way that appears valid to a newly joining node.

To mitigate this problem, Algorand [CM19] and Ouroboros [KRDO17, DGKR18, BGK+18] make use of forward-secure digital signatures [BM99, IR01, DGNW20, WTW+24] to prevent the adversary from forging signatures over past time periods after corrupting a node and obtaining its private state.

Dynamic Availability and Non-Reconfiguration (Sleepy Model). These systems operate from the same fixed set of nodes throughout the entire execution, but nodes may "go to sleep" for an arbitrary amount of time. As such, nodes that are awake must maintain liveness without the presence of all nodes. The sleepy model, therefore, assumes synchrony because it is impossible to solve consensus in the sleepy model if the network can be partitioned [NTT21, LR23].

Sleepy consensus protocols [PS17, MR22] assume an honest majority in the sense that awake and honest nodes outnumber adversarial nodes at any given time. Adversarial nodes are assumed to always be awake. Therefore, as participation grows, the adversary may corrupt more nodes. This becomes increasingly problematic since the adversary may *simulate* as if the newly corrupted nodes were already awake in the past. Consider the execution shown in Fig. 1 in the sleepy model (top-right), where there is one awake node at time 0, and three nodes awake at time 2. If we adapted techniques from the static availability setting, certificate sizes would be relative to the number of available nodes at the time of the event.

Thus, similarly to the long-range attack, the adversary may simulate alternative certificates and rewrite history in a way that appears valid to nodes waking up from their sleep—an attack known as *backward simulation* [DPS19]. Forward-secure digital signatures do not resolve the problem in this setting, since, as mentioned in Sec. 1, sleepy nodes cannot perform key evolution. Recent works [DKT21, MR22, DNTT24, DSTZ23] have solved this issue by completely ignoring past messages and only listening to the latest messages on what decisions to follow.

Dynamic Availability and Reconfiguration (DAR). Naturally, establishing a secure protocol in the DAR model inherits all the difficulties from the two prior models: nodes may be asleep, and the membership set is not fixed. Whenever a node is outdated, either from being asleep for some time or newly joining the system, the node must determine the latest state of the system using only the protocol logic and the initial membership set. We call this process *bootstrapping*. After reasoning about the prior two models, one can recognize the main challenge for bootstrapping: *current* active nodes alone dictate the state of the system (cf. Dynamic Availability and Non-Reconfiguration), but nodes bootstrapping into the system must verify *past* reconfigurations to know who to listen to at present (cf. Static Availability and Reconfiguration).

Bitcoin, a longest-chain Proof-of-Work (PoW) protocol, has a simple bootstrapping procedure with minimum assumptions: a newly joining node only requires the protocol logic, the initial genesis block, and connection to at least one honest and up-to-date node. Even if all other connections are to adversarial nodes, the newly joining node can distinguish the true chain by simply following the one with the most work. Our bootstrapping gadget in the DAR model also requires minimum common knowledge and trust assumptions that are conceptually similar to Bitcoin's bootstrapping procedure, though we show that a stronger variant of the honest majority assumption is required for any consensus protocol to be secure in the DAR model.

Organization. In Sec. 2, we outline basic aspects of our model that is used throughout the paper. In Sec. 3.1, we formalize reconfigurable consensus. In Sec. 3.2, we present our bootstrapping gadget for constructing secure atomic broadcast protocols in the DAR model under the SR-HM assumption. We consider signed membership changes (DAR with sign-off model) and give a bootstrapping gadget with a simple and efficient good-case path in Sec. 4.

1.2 Related Work

Reconfiguration has been well studied, first for crash fault-tolerant consensus [LAB+06, LMZ09, SRMJ12] and later for Byzantine consensus [BSA14, DZ22]. Most of these early works focus on the partial synchronous and (hence inevitably) statically available setting. Reconfigurable consensus has seen a resurgence of interest due to the popularity of Proof-of-Stake (PoS) cryptocurrencies. PoS leverages the allocation of currency in the system to signify the voting power of users. PoS consensus is synonymous with reconfigurable consensus since the distribution of stake defines membership for participation.[2] There are many PoS proposals [DPS19, KRDO17, DGKR18, GHM+17, BGK+18, BHK+20, DZ22] used in practice today. Among those, only a few [DPS19, BGK+18, BHK+20] aim to support dynamic availability, and we have already compared them thoroughly in Sec. 1. Other recent works study dynamically available consensus in the non-reconfigurable setting,

[2]In practice, PoS membership is often weighted by stake. This difference mainly pertains to efficiency and not fundamental feasibility.

most notably sleepy consensus [PS17, MR22, MMR23, DNTT24, DSTZ23, ET25] and PoSAT [DKT21].

Lewis-Pye and Roughgarden [BLR24] provide a systematic overview and taxonomy of various models regarding availability and reconfiguration. They also prove feasible and infeasible results in various models. A notable result that is pertinent to our work is that of [BLR24, Theorem 10.1], which states that any protocol in the reconfigurable setting (what they term the *quasi-permissionless* setting) that only uses time-malleable cryptography (e.g., not forward-secure) is vulnerable to long-range attacks. They do not provide any positive result in the DAR model (what they term as *dynamic availability*). We follow them in not calling PoS "permissionless," because newcomer nodes in PoS need to acquire stake from current nodes to join the system. Although unlikely, one could imagine a scenario where initial stakeholders refuse to transfer their stake (at all or to certain individuals). In contrast, PoW protocols are considered permissionless since possession of computation resources is the only barrier to participation.

Forward-secure signatures were first proposed by Anderson [And97] and formalized by Bellare and Miner [BM99] as protection against the *key exposure problem*, where security of a digital signature scheme is lost when the underlying secret (signing) key is compromised. The recent Pixel multi-signature scheme [DGNW20], adopted by the Algorand blockchain [CM19], achieves forward security and practical efficiency.

2 Model Preliminaries

We consider a lock-step synchronous system in which nodes proceed in globally synchronized discrete *rounds* and are assumed to have synchronized clocks. Messages seen by *any* honest (and awake) node by the start of a round, are seen by *all* honest (and awake) nodes by the end of the round. Subject to this constraint, the adversary can selectively reveal messages it sends to honest nodes. We assume nodes have cryptographic identities that are common knowledge. Let $\mathcal{U}$ be the universe of possible nodes throughout the lifetime of the protocol.

Adversary. The adversary is a probabilistic polynomial-time (PPT) algorithm that can fully adaptively *corrupt* nodes and put nodes to *sleep*. Corrupted nodes (also called *adversarial* or *Byzantine*) may deviate from the protocol arbitrarily in a manner coordinated by the adversary. We denote the corrupted nodes at a round t as $\mathcal{A}_t$. Once a node is corrupted, the node will never become honest again, so $\mathcal{A}_t \subseteq \mathcal{A}_{t+1}$. Uncorrupted nodes (called *honest*) follow the protocol when they are not asleep (as discussed below).

Sleepiness. Besides corruption, the adversary can fully adaptively put nodes to *sleep* at each round. *Sleepy* nodes do not receive or send messages and do not execute the protocol. When a node stops being asleep, it receives all the messages it would have received had it not been asleep (i.e., messages are queued while a node is asleep—blockchains readily

implement this through gossip networks), and the node is then in a state of *booting up*. During this time, the node initiates a *bootstrapping* process and it becomes fully *awake* after completing the bootstrapping process. An awake and honest node executes the specified protocol. We denote awake and honest nodes at a round t as $\mathcal{H}_t$. Adversarial nodes are always considered awake and cannot be put to sleep.

2.1 Definitions

Log. We define a *log* as a sequence of values x_i, denoted as $\Lambda = [x_0, x_1, x_2, \dots]$. We say two logs are *compatible* if one is a prefix of the other. Otherwise, we say they are *conflicting*. At every round, each node has an output log.

Atomic Broadcast. An atomic broadcast protocol [CASD95] allows nodes to agree on a log. Nodes may input requests into the protocol at each round. Logs that are output across honest nodes and across time satisfy the following properties:

- *Safety:* If two honest nodes (possibly the same node) at two points in time (possibly the same time) decide logs Λ and Λ', then Λ and Λ' are compatible.
- *Liveness:* If an awake and honest node inputs a request x, then eventually all awake and honest nodes decide a log containing x.

Forward-Secure Signatures. We make use of a forward-secure signature scheme for signing messages in our protocols. *Forward security* ensures unforgeability of messages in the past—prior to key exposure—such that only messages after key exposure are vulnerable. Bellare and Miner's forward-secure signature construction consists of a single public key $\textsc{pk}$, and an initial signing key denoted $\textsc{sk}_0$. They consider T time periods in which the signature scheme is used. The signer uses a different signing key in each period: $\textsc{sk}_1$ in period 1, $\textsc{sk}_2$ in period 2, and so on. At the beginning of period i, $\textsc{sk}_i$ is derived as a one-way function on $\textsc{sk}_{i-1}$. This is the process of *evolving* the key, and, once complete, the signer deletes the previous key $\textsc{sk}_{i-1}$. Thus, an adversary that corrupts the signer during period i cannot forge messages in prior periods. Let $\mathcal{F}_{\text{FS}}$ be a key-evolving signature scheme parametrized by security parameter λ and the total number of time periods T. We define $\mathcal{F}_{\text{FS}}$ with the following four algorithms taken from Bellare and Miner [BM99] for a signer v:

- **Key Generation:** $(v.\textsc{pk}, \textsc{sk}_0) \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Gen}()$. Signer v runs the key generation algorithm to generate a public verification key $\textsc{pk}$ and an initial secret signing key $\textsc{sk}_0$.
- **Key Update:** $\textsc{sk}_{i+1} \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Update}(\textsc{sk}_i)$. Signer v updates its secret key $\textsc{sk}_i$ to $\textsc{sk}_{i+1}$ for the next time period $i+1$. We may also use $\textsc{sk}_{i'} \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Update}(\textsc{sk}_i, i')$ to repeatedly apply update to any time period $i' > i$.

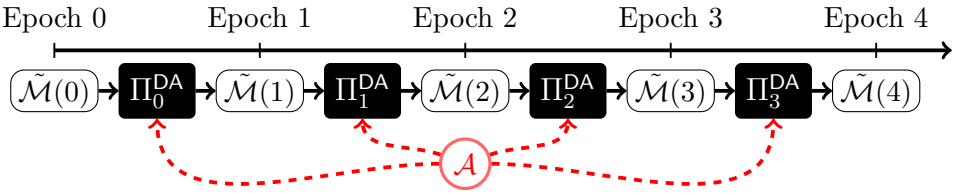

Figure 2: Overview of reconfiguration every epoch. "⇢" indicates the adversary's membership input into Π^{DA}.

- **Sign:** $\langle m \rangle_i^v \leftarrow \mathcal{F}_{\text{FS}}.\text{SIGN}(\text{SK}_i, m)$. Signer v uses the current signing key SK_i and a message m to compute signature $\langle m \rangle_i^v$. For convenience, we assume the signature $\langle m \rangle_i^v$ carries the signer's identity v and the time period i in which the signature is produced.
- **Verification:** $b \leftarrow \mathcal{F}_{\text{FS}}.\text{VERIFY}(\langle m \rangle_i^v)$. Anyone may verify the signature $\langle m \rangle_i^v$ over message m for time period i under the public key $v.\text{PK}$ by running the verification algorithm, which outputs 1 if the signature is valid and 0 otherwise.

As an example, we can use the Pixel forward secure signature [DGNW20], which supports key update, signing, and verification in $O(1)$ cryptographic operations (i.e., group exponentiations and pairings). Both the signature size and public key size are constant. The secret key size is $O(\log^2 T)$ group elements, where T is an upper bound on the number of time periods for which the scheme is used. For instance, if each time period lasts one second, then $T = 2^{32} - 1$ periods would be 136 years, and give a secret key of 43KB in size [DGNW20].

3 Bootstrapping in DAR

This section presents our detailed model for reconfiguration (Sec. 3.1) and our atomic broadcast protocol (Sec. 3.2). We consider a model where reconfiguration requires no action from exiting nodes. We refer to this simply as *the DAR model*. In contrast, in Sec. 4, we will study another variant of the model where exiting nodes must authorize their withdrawal, as is the case in PoS systems.

3.1 Modeling Reconfiguration

The initial membership set $\mathcal{M}_0 \subset \mathcal{U}$ is common knowledge. Subsequent sets $\mathcal{M}_t \subset \mathcal{U}$, for rounds $t > 0$, are not known a priori to all nodes. We assume a fixed membership size, i.e., $|\mathcal{M}_0| = |\mathcal{M}_t|$.

Epochs. Time is divided into fixed-length epochs of R rounds, a parameter chosen beforehand that is common knowledge. Membership is fixed during each epoch and may

change across epochs. Denote the *decided* membership for epoch e as $\tilde{\mathcal{M}}(e)$. Then, we have $\tilde{\mathcal{M}}(e) = \mathcal{M}_{eR} = \mathcal{M}_{eR+1} = \cdots = \mathcal{M}_{eR+R-1}$.

Let Π_e^{DA} be the instance of a dynamically available, non-reconfigurable atomic broadcast protocol associated with epoch e, which is run by nodes in $\tilde{\mathcal{M}}(e)$. At each epoch $e \geq 0$, the adversary may select a new membership set $\mathsf{M} \subset \mathcal{U}$ and input it into the atomic broadcast protocol Π_e^{DA} through its own input channel. At the end of the epoch, the latest decided membership set in the log of honest nodes is set as $\tilde{\mathcal{M}}(e+1)$ and is used to determine membership for the next epoch. We illustrate the process in Fig. 2.

The liveness of Π_e^{DA} ensures that $\tilde{\mathcal{M}}(e+1)$ is *eventually* included in the logs of all awake and honest nodes. However, liveness alone does not guarantee that $\tilde{\mathcal{M}}(e+1)$ is included *by all awake and honest nodes by the end of epoch* e. Specifically, the concern is that, at the end of epoch e, some honest nodes have $\tilde{\mathcal{M}}(e+1)$ in their output log, while other honest nodes do not. Thus, we require an additional property of Π_e^{DA} that ensures the membership set $\tilde{\mathcal{M}}(e+1)$ is in the decided logs of all awake and honest nodes at the end of epoch e. Such a property is easily attainable for common atomic broadcast implementations with simple transformations, albeit with a slight reduction in performance. Given a dynamically available protocol with a worst-case latency of k rounds, a possible transformation is to require "empty proposals" during the last k rounds of the epoch. The last k rounds do not contribute new inputs, decreasing the protocol's throughput. However, it provides adequate time for all awake and honest nodes to settle on a common output by the end of the epoch. Nodes then wait to apply any proposed membership set until the end of the epoch. Under this model, we define the standard honest majority of a schedule (see Def. 1) in Def. 2, which we show at the end of this section is insufficient to guarantee safety and liveness of an atomic broadcast protocol in the DAR model.

Definition 1 (Schedule)**.** *A schedule* K *defines the sequences of sets* $\mathcal{A}_t$*,* $\mathcal{H}_t$*, and* $\mathcal{M}_t$ *of adversarial nodes, awake and honest nodes, and membership at every round* $t \geq 0$*. Note that the corruption, sleep, and membership schedule is chosen by the adversary.*

Definition 2 (Honest Majority (HM))**.** *A schedule satisfies* ***honest majority*** *if for every round* $t \geq 0$*,*

$$|\mathcal{M}_t \cap \mathcal{A}_t| < |\mathcal{M}_t \cap \mathcal{H}_t|.$$

Simulation. To capture what is sufficient and necessary for solving consensus in the DAR model, we pivot to the notion of *simulation*. A key challenge in the dynamically available setting is handling nodes that may have been asleep for arbitrary durations. While asleep, a node does not perform protocol actions: it neither sends nor receives messages, nor evolves its keys. If such a node later becomes corrupted, the adversary can retroactively *simulate* their behavior, making it appear as though they had actively participated in past rounds despite being asleep. We call such nodes *simulatable*. Note that a node is only simulatable up to the last time it was awake and honest, since it may have performed a key evolution at that point. Def. 3 gives the precise definition of simulatable nodes.

Definition 3 (Simulatable nodes)**.** *Denote the set of nodes that have been awake and honest in any round during the range of rounds* $[t_0, t]$ *as*

$$\mathcal{H}(t_0, t) \triangleq \bigcup_{t' \in [t_0, t]} \mathcal{H}_{t'}$$

Then, denote the set of nodes that are simulatable at round t_0 *by the adversary at round* t *as*

$$\mathcal{S}(t_0; t) \triangleq \mathcal{A}_t \setminus \mathcal{H}(t_0, t).$$

Intuitively, the standard honest majority fails at a time t' because the adversary can perform backward simulation from a later time $t > t'$ by leveraging simulatable and adversarial nodes at time t'. To address this gap, we introduce the *simulation-resistant honest majority* in Def. 4, which strengthens the standard honest majority assumption.

Definition 4 (Simulation-Resistant Honest Majority (SR-HM))**.** *A schedule (Def. 1) satisfies* ***simulation-resistant honest majority*** *if for all rounds* $t \geq 0$ *and* $t' \leq t$*, the adversarial and simulatable nodes at round* t' *are less than the awake and honest nodes since round* t'*, i.e.,*

$$\left|\mathcal{M}_{t'} \cap \mathcal{S}(t'; t)\right| < \left|\mathcal{M}_{t'} \cap \mathcal{H}(t', t)\right|.$$

3.2 Bootstrapping Gadget without Sign-Off

In this section, we present an atomic broadcast protocol in the DAR model under SR-HM by layering a bootstrapping gadget atop the dynamically available, non-reconfigurable protocol Π^{DA}. The bootstrapping gadget enables outdated nodes to learn the current membership set and join the current epoch's instance of Π^{DA}. At the end of each epoch, the gadget has awake nodes produce certification votes for the latest decided log, which also behaves as a vote for every prefix of the log. Booting nodes collect these votes upon awakening and follow the log with the most votes from their last known membership set at each epoch until they reach the latest epoch. Although the decided log at each epoch may not have an absolute quorum of votes due to dynamic availability, we prove it safe to follow the log with the "heaviest votes" under SR-HM, since, for every round, the number of awake and honest members is greater than the number of simulatable members. The bootstrapping gadget is detailed in Alg. 1.

Awake Participation. Awake nodes send inputs to Π_e^{DA} during epoch e and output the decided log (ln. 2). At the end of the epoch, nodes produce a forward-secure signature over the log output Λ to send out as a *vote* over the decided log (lns. 4 and 5). Then, nodes obtain the next membership set from Λ, denoted as M_{e+1} for membership at epoch $e+1$ (ln. 6). Lastly, awake nodes evolve their secret keys to the next epoch (ln. 7).

Algorithm 1 Reconfigurable atomic broadcast for node v

1 **during** epoch e, **if** awake
2 Execute Π^{DA} among members in M_e and store output in Λ
3 **at** the last round of epoch e
4 $\langle \Lambda \rangle_e^v \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Sign}(\text{SK}_e, \Lambda)$ // Sign with forward-secure signature
5 Send $(\mathsf{LVOTE}, \langle \Lambda \rangle_e^v)$ to all // Send out vote for latest log
6 $\mathsf{M}_{e+1} \leftarrow \Lambda.\textsc{newMembers}()$ // Get membership set for next epoch from log
7 $\text{SK}_{e+1} \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Update}(\text{SK}_e)$ // Evolve secret key to latest epoch
8 **upon** boot up during epoch e
9 **if** first time booting up
10 $(\text{SK}_0, \text{PK}) \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Gen}()$ // Generate keys
11 $\mathsf{M}_0 \leftarrow \mathcal{M}_0$ // Initialize as initial membership set
12 $\ell \leftarrow e$
13 **else**
14 $\ell \leftarrow$ last epoch v was awake
15 Receive LVOTE messages from network with valid forward-secure signatures
16 **for** $e' \leftarrow \ell, \ell+1, \ldots, e-1$
17 $\mathsf{votes} \leftarrow$ oldest LVOTE messages at or after epoch e' from members in $\mathsf{M}_{e'}$
18 $\Lambda_{e'} \leftarrow$ most voted log up to epoch e' in votes
19 $\mathsf{M}_{e'+1} \leftarrow \Lambda_{e'}.\textsc{newMembers}()$ // Get membership set for next epoch from log
20 $\text{SK}_e \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Update}(\text{SK}_\ell, e)$

Bootstrapping. The boot-up process starts at ln. 8 during an epoch e for a node v. If this is the first time v is awake, it initializes its keys for the forward-secure signature scheme $\mathcal{F}_{\text{FS}}$ and sets its last known membership set to the initial membership $\mathcal{M}_0$. The booting node v then receives all LVOTE messages and verifies the messages for valid signatures (ln. 15). The bootstrapping process begins from v's last known membership set M_ℓ at epoch ℓ. For every epoch $e' \in [\ell, e)$, v collects the oldest votes at or after epoch e' for logs from nodes in $\mathsf{M}_{e'}$ as the set votes (ln. 17). We are interested in the oldest vote since an honest node may be corrupted after issuing a vote, and we want to ensure that we capture all honest votes concerning epoch e'. At ln. 18, v tallies the number of votes for logs up to epoch e' and selects the log with the most votes as $\Lambda_{e'}$. Then, v extracts the next membership set $\mathsf{M}_{e'+1}$ from $\Lambda_{e'}$ (ln. 19). After iterating through every epoch, v has identified the latest membership set and joins the atomic broadcast protocol Π_e^{DA}.

Theorem 3.1. *Under the SR-HM assumption (Def. 4), the protocol shown in Alg. 1 implements atomic broadcast in the DAR model.*

Proof of Thm. 3.1. For every epoch e, since the membership is fixed for the entire epoch, the protocol is safe and live for awake and honest nodes due to the safety and liveness of Π_e^{DA}. It only remains to show that if an honest node v booting up during epoch e participates, it does so with the same membership set $\tilde{\mathcal{M}}(e)$ as the awake and honest nodes in that epoch.

We will inductively show that lns. 8 to 19 identify the decided membership sets from 0 to e. Let t be the round in epoch e in which node v is booting up. For the base case, we have that the initial membership set $\mathcal{M}_0$ is common knowledge. For the inductive step, assume that v has selected the decided membership set $\tilde{\mathcal{M}}(e')$ for epoch $e' < e$. We now show that v selects $\tilde{\mathcal{M}}(e'+1)$ as the confirmed membership set for epoch $e'+1$ at ln. 19.

Consider a false log $\widehat{\Lambda}$ that is conflicting with the decided log $\Lambda_{e'}$. Let c and $\widehat{c}$ be the total count of votes for $\Lambda_{e'}$ and $\widehat{\Lambda}$, respectively, in votes (ln. 17). Let t' be the last round of epoch e'. By the induction hypothesis, all observed votes from round t' are from members in $\tilde{\mathcal{M}}(e')$. Thus, we must have that $c \geq |\mathcal{M}_{t'} \cap \mathcal{H}(t', t)|$, because every honest member in $\mathcal{M}_{t'}$ that was awake at some round after t' signed and sent out a vote for a log that extends $\Lambda_{e'}$. We also have that $\widehat{c} \leq |\mathcal{M}_t \cap (\mathcal{A}_{t'} \cup \mathcal{S}(t'; t))|$ since, by the definition of the forward-secure signature scheme, only adversarial and simulatable nodes at round t' have the secret keys available to sign a vote for a false log. By the SR-HM assumption (Def. 4), we have that

$$\widehat{c} \leq |\mathcal{M}_{t'} \cap \mathcal{S}(t'; t)| < |\mathcal{M}_{t'} \cap \mathcal{H}(t', t, |) \leq c.$$

Thus, we must have that the most voted log for epoch e' is $\Lambda_{e'}$, which v uses to retrieve the next decided membership set $\tilde{\mathcal{M}}(e'+1)$. By induction, we have that node v identifies the latest membership set $\mathsf{M}_e = \tilde{\mathcal{M}}(e)$. □

3.3 Necessity of Simulation-Resistant Honest Majority

We now show that SR-HM is necessary for any atomic broadcast protocol to be safe and live in the DAR model. Importantly, we restrict our attention to *identity-agnostic* conditions, meaning that relabeling nodes in a schedule does not change whether the condition holds. We prove that no *identity-agnostic* condition that fails to imply SR-HM can suffice for an atomic broadcast protocol. Note that both the HM and SR-HM assumptions are identity-agnostic conditions.

Definition 5 (Identity-Permuted Schedule)**.** *For a schedule* K *and a permutation* $\phi \colon \mathcal{U} \to \mathcal{U}$*, define the* identity-permuted schedule $\phi(\mathsf{K})$ *to be the schedule in which node* $\phi(v)$ *inherits the full local view and state (i.e., corruption, honesty, and membership) of node* v *across all rounds in* K*. We say two schedules* K *and* K' *are identity-permuted schedules if there exists a permutation* ϕ *such that* $\mathsf{K}' = \phi(\mathsf{K})$*.*

Definition 6 (Identity-Agnostic Condition)**.** *A condition* C *over schedules is said to be* identity-agnostic *if for all schedule* K*, whenever* C *holds for* K*,* C *also holds for all permuted schedules of* K*, i.e. for all schedule* K *and permutation* ϕ*, we have that*

$$\mathsf{C}(\mathsf{K}) \iff \mathsf{C}(\phi(\mathsf{K})).$$

Definition 7 (Execution)**.** *We define an* execution X *as a sequence of messages and states of all nodes in* $\mathcal{U}$ *under a schedule* K *during execution of a protocol* Π*.*

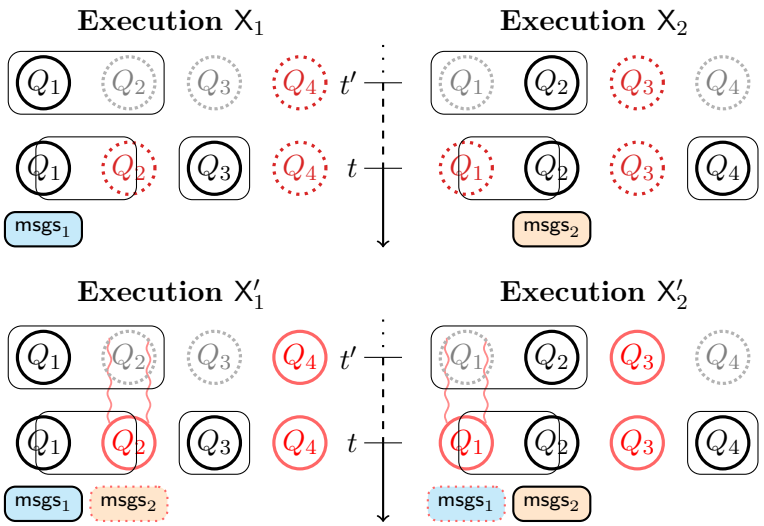

Figure 3: Four executions for possible schedules K_1 and K_2 under condition C. A node booting up in executions X_1 and X'_1 (left) expects to output Λ^1. A node booting up in executions X_2 and X'_2 (right) expects to output Λ^2. Recall, "○" depicts awake and honest nodes, "○" are Byzantine nodes, "○" are sleepy nodes, and "≈" depicts backward simulation.

Theorem 3.2. *No atomic broadcast protocol can be safe and live in the DAR model under an identity-agnostic condition (Def. 6) that does not imply the SR-HM assumption (Def. 4).*

Proof of Thm. 3.2. Let C be an identity-agnostic condition that does not imply SR-HM. For the sake of a contradiction, suppose that Π is an atomic broadcast protocol that satisfies both safety and liveness in the DAR model under C. Since C does not imply SR-HM, the adversary can find a schedule K_1 with rounds $t \geq 0$ and $t' \leq t$ where SR-HM does not hold, i.e.,

$$\left|\mathcal{M}_{t'} \cap \mathcal{S}(t';t)\right| \geq \left|\mathcal{M}_{t'} \cap \mathcal{H}(t',t)\right|.$$

Let $Q_1 = \mathcal{M}_{t'} \cap \mathcal{H}(t',t)$ and $Q_2 \subseteq \mathcal{M}_{t'} \cap \mathcal{S}(t';t)$ such that $|Q_1| = |Q_2|$. Let $Q_3 = \bigcup_{i\in(t',t]} \mathcal{M}_i \setminus \mathcal{M}_{t'}$ be the new set of members from round $t'+1$ to t and let $Q_4 \subset \mathcal{A}_t$ such that $|Q_3| = |Q_4|$ and Q_4 is disjoint from Q_2. This allows us to define a permutation ϕ (Def. 5) that swaps the nodes in Q_1 with those in Q_2 and the nodes Q_3 with those in Q_4, while leaving the remaining nodes unchanged. Because C is identity-agnostic, it also holds for $\mathsf{K}_2 = \phi(\mathsf{K}_1)$.

We now describe executions X_1 and X'_1 with schedule K_1 and executions X_2 and X'_2 with schedule K_2. Fig. 3 illustrates the executions for two example schedules. We have all adversarial nodes in execution X_1 and X_2 behave asleep. Then, let msgs_1 be the set of

messages sent by nodes in $Q_1 \cup Q_3$ during X_1, and msgs_2 be the messages sent by $Q_2 \cup Q_4$ during X_2. Let Λ_1 be the decided log of nodes in $Q_1 \cup Q_3$ at round t during X_1, and let Λ_2 be the decided log of nodes in $Q_2 \cup Q_4$ at round t during X_2 such that Λ_2 conflicts with Λ_1. For nodes not in the sets Q_1, Q_2, Q_3, or Q_4, we have them behave identically in both executions. Let msgs^* be the set of messages sent by these nodes. For executions X'_1 and X'_2 the adversary behaves in the following manner for each execution.

Execution X'_1. By round t, the nodes in $Q_2 \cup Q_4$ must be simulatable at rounds $t', t'+1, \ldots, t$. They then send the same set of messages msgs_2 sent by the awake and honest nodes in X_2, simulating a past view in this execution in which they were awake. By round t, nodes decide on log Λ_1. An honest booting node v will then receive messages $\mathsf{msgs}^* \cup \mathsf{msgs}_1 \cup \mathsf{msgs}_2$ and, by safety of Π, will output log Λ_1.

Execution X'_2. The execution proceeds symmetrically to X'_1 where nodes in $Q_1 \cup Q_3$ are simulatable at rounds $t', t'+1, \ldots, t$. They send the same set of messages msgs_1 sent by the awake and honest nodes in X_1. By round t, nodes decide on a log Λ_2 that conflicts with Λ_1. An honest booting node v will then receive messages $\mathsf{msgs}^* \cup \mathsf{msgs}_1 \cup \mathsf{msgs}_2$, and by safety of Π, will output log Λ_2.

Contradiction. The booting node v receives the same set of messages $\mathsf{msgs}^* \cup \mathsf{msgs}_1 \cup \mathsf{msgs}_2$ in both executions X'_1 and X'_2. Therefore, v cannot distinguish between the two executions, and we have a contradiction: either v outputs Λ_2 in X'_1 and violates safety, or v outputs Λ_1 in X'_2 and violates safety. Thus, no atomic broadcast protocol can be safe and live for any identity-agnostic condition that does not imply the SR-HM assumption (Def. 4). □

4 Bootstrapping in DAR with Sign-Off

We now consider a setting where nodes get to execute a *sign-off* process before exiting the membership set. This aligns with PoS systems, where nodes sign transactions to transfer their stake. There are two main reasons why the sign-off process helps in solving the bootstrapping problem. First, nodes can be required to produce a signature over the transaction, binding them to the change. Second, the protocol can require nodes to perform *key disposal*, i.e., delete their private keys, immediately after signing off.[3] We assume even sleepy honest nodes perform key disposal. In contrast to key evolution while asleep, key disposal upon sign-off is a reasonable assumption in PoS systems because, although not participating in the protocol, sleepy nodes must be awake in some capacity to sign the transaction that transfers their stake.

[3]We do consider *rational* nodes that may wish to sell their private keys instead of deleting them. A node not deleting its key when signing off is considered Byzantine.

4.1 Modeling Sign-Off

We now extend the model described in Sec. 3.1 to incorporate sign-off behavior. Recall that during an epoch $e \geq 0$, the adversary may input a membership set into Π_e^{DA}, and the latest decided membership set at the end of the epoch becomes the next membership set $\tilde{\mathcal{M}}(e+1)$. With sign-off, we assume there exists a one-to-one mapping τ of withdrawing nodes in $\tilde{\mathcal{M}}(e)$ to their replacement nodes in $\tilde{\mathcal{M}}(e+1)$. For every honest withdrawing node, the adversary calls their sign-off function with their given replacement node. Formally, for every $(v, v') \in \tau$, $v.\textsc{SignOff}(v')$ is executed by the end of epoch e. Notably, the mapping τ is not input into Π_e^{DA} and is not common knowledge to honest nodes.[4] The sign-off function specifies the sign-off behavior of the protocol, such as signing a transaction and key disposal. The withdrawing node v, whether awake or not, is assumed to know the current epoch e and to be connected to at least one awake and honest node in $\tilde{\mathcal{M}}(e)$.

Disposal. With a weaker adversary in the DAR with sign-off model, we may weaken the SR-HM assumption. In particular, consider that the adversary may no longer simulate past behaviors of withdrawn sleepy nodes due to irreversible changes during the sign-off process, such as key disposal. We say a node is *disposed* if it has executed its sign-off function. Only corrupted nodes may avoid being disposed of among nodes that have withdrawn. Therefore, we define the set of simulatable nodes in the DAR with sign-off model in Def. 8.

Definition 8 (Simulatable Nodes in the sign-off model)**.** *Denote the set of nodes that were honest and awake or withdrew from a membership set during the range of rounds* $[t_0, t)$ *as*

$$\mathcal{W}(t_0, t) \triangleq \bigcup_{t' \in [t_0, t)} (\mathcal{M}_{t'} \setminus \mathcal{M}_{t'+1}) \setminus \mathcal{A}_{t'}.$$

Then, denote the set of nodes that are simulatable *(Def. 3) at round* t_0 *by the adversary at round* t *as*

$$\mathcal{S}_{\overline{\mathcal{W}}}(t_0; t) \triangleq (\mathcal{A}_t \setminus \mathcal{H}(t_0, t)) \setminus \mathcal{W}(t_0, t).$$

With the new definition of simulatable nodes in the DAR with sign-off model, the set $\mathcal{S}(t'; t)$ in the SR-HM condition (Def. 4) is replaced by $\mathcal{S}_{\overline{\mathcal{W}}}(t'; t)$ in this section.

4.2 Bootstrapping Gadget with Sign-Off

In this section, we present our bootstrapping gadget under the SR-HM assumption (Def. 4) among non-disposed simulatable nodes. Our gadget with sign-off leverages the detectability of double-spending and the unforgeability of honest transactions. During periods without

[4] This models a slightly weaker constraint than typical PoS systems that reach consensus on each transaction for stake changes.

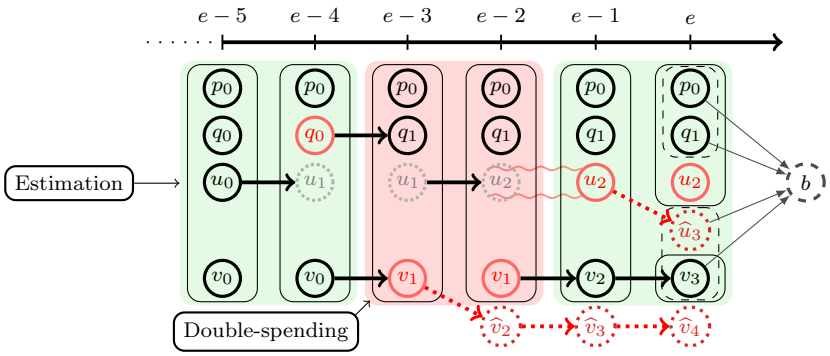

Figure 4: Example of estimating membership from the view of a booting node b. Here, "$\longrightarrow$" are decided transactions, "$\dashrightarrow$" are undecided transactions, "$\longrightarrow$" are membership votes, and "$\dashrightarrow$" are non-members. The green regions indicate periods of estimation. The red region indicates a period with a double spender (v_1) that requires voting to identify the true membership change. The second estimation period has a *hidden spend* from node u_2 to $\widehat{u}_3$. The dashed borders indicate node b's estimated membership set for epoch e that it listens to for the exact membership set (represented by the solid borders).

double spending, we skip membership voting via a good-case path that constructs membership sets from the transactions received, which we call *membership estimation*. We illustrate the process in Fig. 4.

In many practical systems, it is reasonable to expect that only a small fraction of members are reconfigured in each epoch. In that case, there will be much fewer membership-changing transactions than membership votes, so the good-case estimation allows far more efficient bootstrapping than following voting.

Estimation allows a booting node to never miss a decided transaction, but a subtle problem emerges when undecided transactions from the adversary are included. We call such transactions *hidden spends* as shown in Fig. 4. Such hidden spends may cause a booting node to use a disjoint membership set in the atomic broadcast protocol Π^{DA}. Therefore, awake and honest nodes send out a vote for the current membership set in every round. Once a booting node finishes estimation up to the latest epoch, the node then follows the set with the most votes from members in the estimated sets, which is distinguishable due to an honest majority in the estimated set. We give our bootstrapping gadget in Alg. 2.

Awake Execution. Awake nodes follow the same steps as the previous gadget by using Π_e^{DA} among the latest membership set and signing votes for the log outputted by Π_e^{DA} using the forward-secure signature scheme. In addition, awake nodes send votes for the current membership and disseminate transactions at each round in an epoch at lns. 2 to 5.

Algorithm 2 Reconfigurable atomic broadcast with bootstrapping gadget for node v

1 **during** epoch e, **if** awake
2 **for** every round in e except the last
3 $\langle \mathsf{M}_e \rangle_e^v \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Sign}(\text{SK}_e, \mathsf{M}_e)$
4 Send $(\mathsf{MVOTE}, \langle \mathsf{M}_e \rangle_e^v)$ to all
5 Send transactions received to all
6 same as lns. 2 to 7 in Alg. 1
7 **upon** boot up during epoch e
8 same as lns. 9 to 14 in Alg. 1
9 Receive $\mathsf{MVOTE}, \mathsf{LVOTE}$, and TX messages with valid forward-secure signatures
10 $\mathcal{D} \leftarrow$ nodes that have signed transactions for conflicting receivers
11 $\mathsf{E}_\ell \leftarrow \mathsf{M}_\ell$ // Initialize estimated membership to last known membership set
12 **for** $e' \leftarrow \ell, \ell+1, e-1$
13 $\mathcal{T}_{e'} \leftarrow \mathsf{TX}$ messages signed during epoch e' from members in $\mathsf{E}_{e'}$
14 **if** there exists a transaction from a double spender in $\mathcal{T}_{e'}$
15 $\mathsf{votes} \leftarrow$ oldest LVOTE messages at or after epoch e' from members in $\mathsf{E}_{e'}$
16 $\Lambda_{e'} \leftarrow$ most voted log up to epoch e' in votes
17 $\mathsf{E}_{e'+1} \leftarrow \Lambda_{e'}.\textsc{newMembers}()$ // Get membership for next epoch from epoch e' log
18 **else** // Otherwise directly apply $\mathcal{T}_{e'}$
19 $\mathsf{E}_{e'+1} \leftarrow \textsc{ApplyTxs}(\mathsf{E}_{e'}, \mathcal{T}_{e'})$
20 $\mathsf{votes} \leftarrow \mathsf{MVOTE}$ messages with signatures for epoch e from members in E_e
21 $\mathsf{M}_e \leftarrow$ most voted membership set in votes
22 $\text{SK}_e \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Update}(\text{SK}_\ell, e)$ // Evolve secret key to latest epoch
23 **procedure** $\textsc{SignOff}(v')$ // Sign-off procedure when called during an epoch e
24 $\ell \leftarrow$ last epoch v was awake
25 $\text{SK}_e \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Update}(\text{SK}_\ell, e)$
26 $\langle (v, v') \rangle_e^v \leftarrow \mathcal{F}_{\text{FS}}.\textsc{Sign}(\text{SK}_e, (v, v'))$
27 Delete SK_e
28 Send $(\mathsf{TX}, \langle (v, v') \rangle_e^v)$ to all

Bootstrapping. The boot-up process starts at ln. 7 when a node v boots up during an epoch e. When v wakes up for the first time, it follows the same initialization steps as the previous gadget. Then, v receives all valid votes and transactions signed with forward-secure signatures from the network at ln. 9. For every epoch e', v filters for all the transactions $\mathcal{T}_{e'}$ that were signed for epoch e' from its last estimated membership set (ln. 13). If there exist conflicting transactions in $\mathcal{T}_e$, v follows the membership set with the "heaviest votes" (lns. 14 to 17) similar to the prior gadget.

In the absence of conflicting transactions in $\mathcal{T}_e$, v applies all transactions in $\mathcal{T}_e$ onto its last estimated membership set (ln. 19). Once v has a membership set up to the latest epoch e, v identifies the decided membership set by following the set with "heaviest votes" among its estimated members (lns. 20 and 21).

Theorem 4.1. *Under the SR-HM assumption (Def. 4), the protocol shown in Alg. 2 implements atomic broadcast in the DAR with sign-off model.*

Proof of Thm. 4.1. For every epoch e, the protocol is safe and live for awake and honest nodes due to the safety and liveness of Π^{DA}. It remains to show that an honest node v booting up during epoch e participates with the same membership set $\tilde{\mathcal{M}}(e)$ as the awake and honest nodes in that epoch.

Definition 9 (Transactions constructing a membership set)**.** *Let $\mathcal{T}$ be all the valid transactions v has received. We say a set of transactions $\mathcal{T}' \subseteq \mathcal{T}$* ***constructs*** *a membership set M if $\textsc{ApplyTxs}(\mathcal{M}_0, \mathcal{T}') = \mathsf{M}$. Let $\mathsf{M}.\mathsf{txs}$ denote the transactions that construct M.*

Definition 10 (Conflicting Transactions)**.** *Let $\mathcal{T}(\mathcal{D}) \subset \mathcal{T}$ be the set of transactions observed from double spenders (ln. 10). We denote $\mathcal{C} \subseteq \mathcal{T}(\mathcal{D})$ as the set of conflicting transactions from double spenders that were not decided, i.e.*

$$\mathcal{C} \triangleq \mathcal{T}(\mathcal{D}) \setminus \tilde{\mathcal{M}}(e).\mathsf{txs}.$$

See Fig. 4 for an example of a conflicting transaction from v_1 to $\widehat{v}_2$.

Definition 11 (Estimated Membership Set)**.** *We say a set of nodes $\mathsf{E}_{e'} \subset \mathcal{U}$ is an estimated membership set for an epoch $e' \leq e$ if it is constructed from a set of non-conflicting transactions that includes all decided transactions up to epoch e', i.e. such that*

$$\tilde{\mathcal{M}}(e').\mathsf{txs} \subseteq \mathsf{E}_{e'}.\mathsf{txs} \subseteq \mathcal{T} \setminus \mathcal{C}$$

We will prove by induction that the bootstrapping process defined by lns. 7 to 22 identifies *estimated membership sets* from epoch 0 to e. For the base case, the initial membership set $\mathcal{M}_0$ is common knowledge and, by definition, is an estimated membership set. For the inductive step, assume that v has identified an estimated membership set $\mathsf{E}_{e'}$ for an epoch $e' < e$. We now show that the next membership set $\mathsf{E}_{e'+1}$ for v is an estimated membership set for epoch $e' + 1$.

Let $\mathcal{T}_{e'}$ be the set of valid transactions for epoch e' from nodes in $\mathsf{E}_{e'}$ as defined by ln. 13. Based on node v's view, there are two cases to consider: whether or not $\mathcal{T}_{e'}$ contains a transaction from a double spender.

Double Spender. In this case, the condition at ln. 14 is true, and node v proceeds into a voting-based procedure similar to our prior bootstrapping gadget in Sec. 3.2. Consider log $\widehat{\Lambda}$ that is conflicting with the decided log $\Lambda_{e'}$ for epoch e'. Let c and $\widehat{c}$ be the total count of votes for $\Lambda_{e'}$ and $\widehat{\Lambda}$, respectively, in votes (ln. 15). Let t' be the last round of epoch e', and t be the round node v is booting up in. We first observe that honest nodes that evolved or disposed of their keys must be in the estimated set. Notably, such nodes include all honest nodes, so we must have that $c \geq |\mathcal{M}_{t'} \cap \mathcal{H}(t', t)|$ since every honest member in $\mathcal{M}_{t'}$ that was awake some round after t' signed and sent out a vote for a log that extends $\Lambda_{e'}$.

Any members in $\mathsf{E}_{e'}$ that are not in $\tilde{\mathcal{M}}(e')$ are constructed from undecided transactions in $\mathsf{E}_{e'}.\mathsf{txs}$. These members cannot be more than the adversarial and non-disposed simulatable nodes in $\tilde{\mathcal{M}}(e')$. Then, we must have that the number of votes for $\widehat{\mathsf{M}}$ is at most $\widehat{c} < |\mathcal{M}_{t'} \cap (\mathcal{A}_{t'} \cup \mathcal{S}_{\overline{\mathcal{W}}}(t'; t))|$. By the SR-HM assumption (Def. 4) among non-disposed simulatable nodes, we then have

$$\widehat{c} < |\mathcal{M}_{t'} \cap \mathcal{S}_{\overline{\mathcal{W}}}(t'; t)| < |\mathcal{M}_{t'} \cap \mathcal{H}(t', t)| \leq c.$$

Thus, it must be that $\Lambda_{e'}$ is the most voted membership set, which v uses to retrieve the next decided membership set $\tilde{\mathcal{M}}(e'+1)$.

No Double Spender. In this case, $\mathsf{E}_{e'+1}$ is directly constructed from $\mathcal{T}_{e'}$ over $\mathsf{E}_{e'}$ at ln. 19. We show that $\mathcal{T}_{e'}$ must contain all the decided transactions from epoch e'. Consider a sender q of a decided transaction in epoch e', who must be a member in $\tilde{\mathcal{M}}(e')$. The sender q must also remain a member in $\mathsf{E}_{e'}$ since q is not a double spender, implying no existing transaction of q exists in $\mathsf{E}_{e'}.\mathsf{txs}$. Thus, $\mathsf{E}_{e'+1}$ is an estimated set since it is constructed from all the decided transactions up to epoch e' and does not contain any conflicting transactions.

By induction, the booting node v identifies estimated membership sets from epoch 0 to epoch e. Let E_e be the estimated membership set for epoch e. We now argue that v identifies the decided membership set $\tilde{\mathcal{M}}(e)$. Let $\widehat{\mathsf{M}} \neq \tilde{\mathcal{M}}(e)$ be an undecided membership set for epoch e, and let c and $\widehat{c}$ be the number of votes received for $\tilde{\mathcal{M}}(e)$ and $\widehat{\mathsf{M}}$, respectively, at ln. 20. We must have that E_e includes all honest members in the latest round t in which v is booting up in since the adversary cannot forge their signatures and create undecided transactions for them, i.e. $\mathsf{E}_e \supseteq \mathcal{M}_t \setminus \mathcal{A}_t$. Thus, we must have that $c \geq |\mathcal{H}_t|$. Furthermore, since $|\mathsf{E}_e| = |\mathcal{M}_t| = |\mathcal{M}_t \setminus \mathcal{A}_t| + |\mathcal{M}_t \cap \mathcal{A}_t|$, it must be that E_e has at most $|\mathcal{M}_t \cap \mathcal{A}_t|$ adversarial nodes that can sign votes, implying $\widehat{c} \leq |\mathcal{M}_t \cap \mathcal{A}_t|$. Therefore, by the SR-HM assumption (Def. 4), we must have that

$$\widehat{c} \leq |\mathcal{M}_t \cap \mathcal{A}_t| < |\mathcal{M}_t \cap \mathcal{H}_t| \leq c.$$

Thus, $\tilde{\mathcal{M}}(e)$ is the most voted membership set, and node v stores it in M_e at ln. 21. Node v can then participate under the same configuration of Π^{DA} as the latest awake and honest nodes. Therefore, we have that Alg. 2 is a safe and live atomic broadcast protocol under the DAR with sign-off model. □

5 Conclusion

We studied the fundamental limits of achieving consensus under the realistic setting of Dynamic Availability and Reconfiguration (DAR). We showed that in the absence of any hard-to-justify assumptions, consensus is only achievable under a strict *simulation-resistant honest majority* (Def. 4). We formalized this barrier with a tight lower bound and

complemented it with a generic bootstrapping gadget that safely extends any dynamically available, non-reconfigurable protocol (i.e., the sleepy model) to the DAR setting under SR-HM. Our matching bounds characterize precisely, under what circumstances, consensus is feasible in the DAR setting without additional assumptions. We then introduce a variant of the DAR model with a sign-off requirement—typically satisfied in PoS systems. In the DAR with sign-off model, we can leverage two additional facts: honest withdrawals are unforgeable (even for sleepy nodes that are later corrupted) due to key disposal, and double-spending is detectable due to signatures and network synchrony. We presented a second bootstrapping gadget that has a more efficient good-case path of *membership estimation* with a fallback to voting-based resolution.

Acknowledgments. This work is funded in part by the National Science Foundation award #2143058.

References

[ADKS22] Ittai Abraham, Danny Dolev, Alon Kagan, and Gilad Stern. Brief announcement: Authenticated consensus in synchronous systems with mixed faults. In *DISC*, volume 246 of *LIPIcs*, pages 38:1–38:3. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 2022.

[And97] Ross Anderson. Two remarks on public key cryptology. *Invited Lecture, ACM CCS'97*, 1997. `https://www.cl.cam.ac.uk/techreports/UCAM-CL-TR-549.pdf`.

[BGK+18] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In *CCS*, pages 913–930. ACM, 2018.

[BHK+20] Vitalik Buterin, Diego Hernandez, Thor Kamphefner, Khiem Pham, Zhi Qiao, Danny Ryan, Juhyeok Sin, Ying Wang, and Yan X Zhang. Combining ghost and casper, 2020.

[BLR24] Eric Budish, Andrew Lewis-Pye, and Tim Roughgarden. The economic limits of permissionless consensus. In *EC*, pages 704–731. ACM, 2024.

[BM99] Mihir Bellare and Sara K. Miner. A forward-secure digital signature scheme. In *CRYPTO*, volume 1666 of *LNCS*, pages 431–448. Springer, 1999.

[BSA14] Alysson Neves Bessani, Joao Sousa, and Eduardo Adílio Pelinson Alchieri. State machine replication for the masses with BFT-SMART. In *DSN*, pages 355–362. IEEE Computer Society, 2014.

[CASD95] Flaviu Cristian, Houtan Aghili, H. Raymond Strong, and Danny Dolev. Atomic broadcast: From simple message diffusion to byzantine agreement. *Inf. Comput.*, 118(1):158–179, 1995.

[CL99] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *OSDI*, pages 173–186. USENIX Association, 1999.

[CM19] Jing Chen and Silvio Micali. Algorand: A secure and efficient distributed ledger. *Theor. Comput. Sci.*, 777:155–183, 2019.

[CMSK07] Byung-Gon Chun, Petros Maniatis, Scott Shenker, and John Kubiatowicz. Attested append-only memory: making adversaries stick to their word. In *SOSP*, pages 189–204. ACM, 2007.

[DGKR18] Bernardo David, Peter Gazi, Aggelos Kiayias, and Alexander Russell. Ouroboros praos: An adaptively-secure, semi-synchronous proof-of-stake blockchain. In *EUROCRYPT (2)*, volume 10821 of *LNCS*, pages 66–98. Springer, 2018.

[DGNW20] Manu Drijvers, Sergey Gorbunov, Gregory Neven, and Hoeteck Wee. Pixel: Multi-signatures for consensus. In *USENIX Security Symposium*, pages 2093–2110. USENIX Association, 2020.

[DKT21] Soubhik Deb, Sreeram Kannan, and David Tse. Posat: Proof-of-work availability and unpredictability, without the work. In *Financial Cryptography (2)*, volume 12675 of *LNCS*, pages 104–128. Springer, 2021.

[DNTT24] Francesco D'Amato, Joachim Neu, Ertem Nusret Tas, and David Tse. Goldfish: No more attacks on ethereum?! In *FC (1)*, volume 14744 of *LNCS*, pages 3–23. Springer, 2024.

[DPP19] Evangelos Deirmentzoglou, Georgios Papakyriakopoulos, and Constantinos Patsakis. A survey on long-range attacks for proof of stake protocols. *IEEE Access*, 7:28712–28725, 2019.

[DPS19] Phil Daian, Rafael Pass, and Elaine Shi. Snow white: Robustly reconfigurable consensus and applications to provably secure proof of stake. In *Financial Cryptography*, volume 11598 of *LNCS*, pages 23–41. Springer, 2019.

[DSTZ23] Francesco D'Amato, Roberto Saltini, Thanh-Hai Tran, and Luca Zanolini. Tob-svd: Total-order broadcast with single-vote decisions in the sleepy model, 2023.

[DZ22] Sisi Duan and Haibin Zhang. Foundations of dynamic BFT. In *SP*, pages 1317–1334. IEEE, 2022.

[ET25] Yuval Efron and Ertem Nusret Tas. Dynamically available common subset. Cryptology ePrint Archive, Paper 2025/016, 2025.

[GDCL22] Mingyuan Gao, Hung Dang, Ee-Chien Chang, and Jialin Li. Mixed fault tolerance protocols with trusted execution environment, 2022.

[GHM+17] Yossi Gilad, Rotem Hemo, Silvio Micali, Georgios Vlachos, and Nickolai Zeldovich. Algorand: Scaling byzantine agreements for cryptocurrencies. In *SOSP*, pages 51–68. ACM, 2017.

[GP92] Juan A. Garay and Kenneth J. Perry. A continuum of failure models for distributed computing. In *WDAG*, volume 647 of *LNCS*, pages 153–165. Springer, 1992.

[HM84] Joseph Y. Halpern and Yoram Moses. Knowledge and common knowledge in a distributed environment. In *PODC*, pages 50–61. ACM, 1984.

[IR01] Gene Itkis and Leonid Reyzin. Forward-secure signatures with optimal signing and verifying. In *CRYPTO*, volume 2139 of *LNCS*, pages 332–354. Springer, 2001.

[JM14] Leander Jehl and Hein Meling. Asynchronous reconfiguration for paxos state machines. In *ICDCN*, volume 8314 of *LNCS*, pages 119–133. Springer, 2014.

[KRDO17] Aggelos Kiayias, Alexander Russell, Bernardo David, and Roman Oliynykov. Ouroboros: A provably secure proof-of-stake blockchain protocol. In *CRYPTO (1)*, volume 10401 of *LNCS*, pages 357–388. Springer, 2017.

[LAB+06] Jacob R. Lorch, Atul Adya, William J. Bolosky, Ronnie Chaiken, John R. Douceur, and Jon Howell. The SMART way to migrate replicated stateful services. In *EuroSys*, pages 103–115. ACM, 2006.

[LMZ09] Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Vertical paxos and primary-backup replication. In *PODC*, pages 312–313. ACM, 2009.

[LMZ10] Leslie Lamport, Dahlia Malkhi, and Lidong Zhou. Reconfiguring a state machine. *SIGACT News*, 41(1):63–73, 2010.

[LR23] Andrew Lewis-Pye and Tim Roughgarden. Byzantine generals in the permissionless setting. In *FC (1)*, volume 13950 of *LNCS*, pages 21–37. Springer, 2023.

[MMR23] Dahlia Malkhi, Atsuki Momose, and Ling Ren. Towards practical sleepy BFT. In *CCS*, pages 490–503. ACM, 2023.

[MR22] Atsuki Momose and Ling Ren. Constant latency in sleepy consensus. In *CCS*, pages 2295–2308. ACM, 2022.

[Nak09] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system. 2009. `https://bitcoin.org/bitcoin.pdf`.

[NTT21] Joachim Neu, Ertem Nusret Tas, and David Tse. Ebb-and-flow protocols: A resolution of the availability-finality dilemma. In *SP*, pages 446–465. IEEE, 2021.

[OO14] Diego Ongaro and John K. Ousterhout. In search of an understandable consensus algorithm. In *USENIX ATC*, pages 305–319. USENIX Association, 2014.

[PS17] Rafael Pass and Elaine Shi. The sleepy model of consensus. In *ASIACRYPT (2)*, volume 10625 of *LNCS*, pages 380–409. Springer, 2017.

[SRMJ12] Alexander Shraer, Benjamin C. Reed, Dahlia Malkhi, and Flavio Paiva Junqueira. Dynamic reconfiguration of primary/backup clusters. In *USENIX ATC*, pages 425–437. USENIX Association, 2012.

[WTW+24] Jianghong Wei, Guohua Tian, Ding Wang, Fuchun Guo, Willy Susilo, and Xiaofeng Chen. Pixel+ and pixel++: Compact and efficient forward-secure multi-signatures for pos blockchain consensus. In *USENIX Security Symposium*. USENIX Association, 2024.